OurMocks

2027 CYCLE

MOCK · PRACTICE · CRACK

CUET UG

Computer Science

Searching & Hashing Techniques

Linear Search, Binary Search, Modulo Hash Functions, Collision Resolution & CPython Dict

INSIDE
Concept Notes

INSIDE
Solved Examples

INSIDE
Practice MCQs + Solutions

Aligned to the latest official syllabus · Concept-first explanations

www.ourmocks.com

Syllabus Alignment

Section / Unit	Official NTA Syllabus Pointers	Core Focus & High-Yield Exam Traps
Section B1: Topic 3 (Class 12)	Searching Techniques: Linear Search, Binary Search, divide-and-conquer strategy, iterative vs recursive algorithms, time complexity bounds.	Preconditions for Binary Search (strictly sorted arrays with $O(1)$ random access). Midpoint integer overflow trap, comparison counts in best/worst cases.
Section B1: Topic 3 (Class 12)	Hashing Concepts & Hash Functions: Direct access tables, division/modulo method, mid-square, folding, multiplication method.	Properties of ideal hash functions. Choosing prime moduli m, calculating hash bucket addresses mathematically for given integer keys.
Section B1: Topic 3 (Class 12)	Collision Resolution & Load Factor: Open Addressing (Linear Probing, Quadratic, Double Hashing), Separate Chaining, Primary Clustering, CPython dict.	Tombstone markers during deletion in open addressing. Primary clustering hazards, load factor $\lambda = \frac{N}{m}$, rehashing threshold ($\lambda >$ 0.66 to 0.75).

EXAM PATTERN

CUET UG Exam Architecture: Unit 4 (Searching & Hashing) accounts for approximately 8% to 12% of Section B1 in the CUET UG Computer Science paper (4 to 5 MCQs). Core question archetypes include tracing binary search midpoint comparisons on sorted arrays, computing hash table slot allocations under linear probing, calculating load factors after insertions, and evaluating the worst-case complexity of separate chaining vs open addressing.

Concept Module 1: Linear Search (Sequential Search)

1.1 Algorithmic Mechanics & Characteristics

Linear Search (also known as **Sequential Search**) is the most intuitive and foundational search algorithm:

- The algorithm begins at the very first element (index 0) of a collection and inspects each element sequentially one by one until either:
  1. An element matching the search key is found (search succeeds, returning the corresponding index).
  2. The end of the collection is reached without finding a match (search fails, returning -1 or sentinel).
- **Zero Preconditions:** Linear Search requires **no prior ordering** or structural organization of the data. It operates equally on unsorted arrays, linked lists, and streaming data feeds.

```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i  # Target found at index i
    return -1         # Target absent
```

1.2 Complexity Analysis of Linear Search

- **Best-Case Time Complexity:** $O(1)$ constant time. Occurs when the target element resides at the very first position (index 0). Exactly 1 comparison is required.
- **Worst-Case Time Complexity:** $O(N)$ linear time. Occurs when the target element resides at the final position (index $N-1$) or is entirely absent from the list. Exactly N comparisons are executed.
- **Average-Case Time Complexity:** $O(N)$ linear time. Assuming uniform probability of the target existing at any position, the expected number of comparisons is:

$$\text{Average Comparisons} = \frac{1+2+...+N}{N} = \frac{N(N+1)}{2N} = \frac{N+1}{2} \approx \frac{N}{2} = O(N)$$

- **Space Complexity:** Strictly $O(1)$ auxiliary memory (only counter variables are maintained).

Concept Module 2: Binary Search (Divide-and-Conquer)

2.1 The Divide-and-Conquer Principle & Strict Preconditions

Binary Search is a highly efficient search algorithm that operates on the **Divide-and-Conquer** paradigm:

- **Strict Mandatory Precondition:** The collection **must be strictly sorted** in ascending (or descending) order, and the underlying data structure **must support $O(1)$ random access** by index (such as an array or Python list). Binary search cannot be implemented efficiently on singly-linked lists because finding the midpoint takes $O(N)$ time.
- **Search Range Elimination:** In each iteration, Binary Search compares the target with the element residing at the exact midpoint:
  1. If `arr[mid] == target`: The search terminates successfully.
  2. If `arr[mid] > target`: The target must lie in the left half; the right half is discarded by setting `high = mid - 1`.
  3. If `arr[mid] < target`: The target must lie in the right half; the left half is discarded by setting `low = mid + 1`.
- Every comparison halves the remaining search range from N to $\frac{N}{2}$, then $\frac{N}{4}, \frac{N}{8}, ..., 1$.

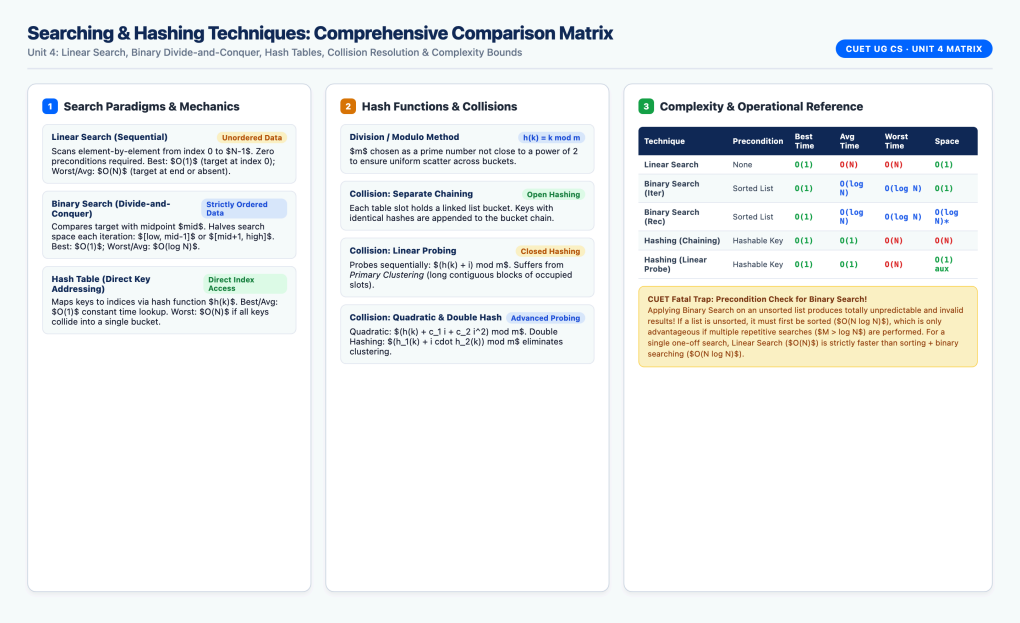


Figure 1: Figure 1: Searching & Hashing Comparison Matrix: Comprehensive analysis of preconditions, algorithmic strategies, best/worst time complexities, and memory requirements.

2.2 Mathematical Complexity & Safe Midpoint Calculation

- **Recurrence Relation:** $T(N) = T\left(\frac{N}{2}\right) + O(1)$. By the Master Theorem, this yields $T(N) = O(\log N)$.
- **Maximum Comparisons:** The maximum number of comparisons in the worst case is strictly:
$$\text{Max Comparisons} = \lfloor \log_2(N) \rfloor + 1$$
  - For $N = 1000$: at most 10 comparisons.
  - For $N = 1,000,000$: at most 20 comparisons.
- **The Midpoint Overflow Trap:** In languages with fixed 32-bit signed integers (C/C++, Java), calculating `mid = (low + high) // 2` can sum `low + high` beyond $2^{\{31\}} - 1$, causing overflow and wrapping to a negative index! The robust, overflow-safe formulation is:
$$\text{mid} = \text{low} + \left\lfloor \frac{\text{high} - \text{low}}{2} \right\rfloor$$

Concept Module 3: Hashing Concepts & Hash Functions

3.1 Direct Access Tables vs Hashing

Direct addressing allocates an array index for every conceivable key. However, if the universe of keys U is enormous (e.g. 10-digit Aadhaar / Phone numbers), direct arrays would require terabytes of unused RAM:

- **Hashing** maps a large universe of keys U into a compact array (Hash Table) of size m using a mathematical mapping function termed a **Hash Function** $h(k)$:
$$h : U \to \{0, 1, 2, ..., m-1\}$$
- An optimal hash table achieves $O(1)$ expected time for insertions, deletions, and key lookups.

3.2 Desirable Properties of an Ideal Hash Function

1. **Deterministic:** The same key k must **always** produce the exact same hash address $h(k)$.
2. **Uniform Distribution:** Keys should scatter evenly across all m slots, minimizing clustering.
3. **Fast Computation:** Must compute in $O(1)$ time with minimal CPU arithmetic cycles.
4. **Minimizes Collisions:** Two distinct keys $k_1 \neq k_2$ should rarely produce $h(k_1) = h(k_2)$.

3.3 Classic Hash Function Methodologies

1. **Division / Modulo Method:**
$$h(k) = k \bmod m$$
   - The table size m should be chosen as a **prime number** not close to a power of 2 or 10. Choosing a prime minimizes common factors between key patterns and table size, reducing clustering.
2. **Mid-Square Method:**

- Square the numeric key (k^2), and extract the middle r digits as the index.
- For example, if $k = 45$, $k^2 = 2025$. Extracting the middle two digits yields $h(k) = 02$.
- Advantage: Every digit of the original key contributes to the middle digits of the square.

3. **Folding Method:**
   - The key k is partitioned into equal-sized numeric chunks (except possibly the last chunk).
   - The chunks are summed together, and the sum is reduced modulo m.
   - Subtypes: **Fold-Shifting** (chunks added directly) vs **Fold-Boundary** (boundary chunks reversed before addition).
4. **Multiplication Method:**

$$h(k) = \lfloor m \cdot ((k \cdot A) \bmod 1) \rfloor$$

   - Where A is a constant fraction in range $0 < A < 1$. Donald Knuth suggested $A = \frac{\sqrt{5}-1}{2} \approx 0.6180339887$ (the Golden Ratio conjugate).

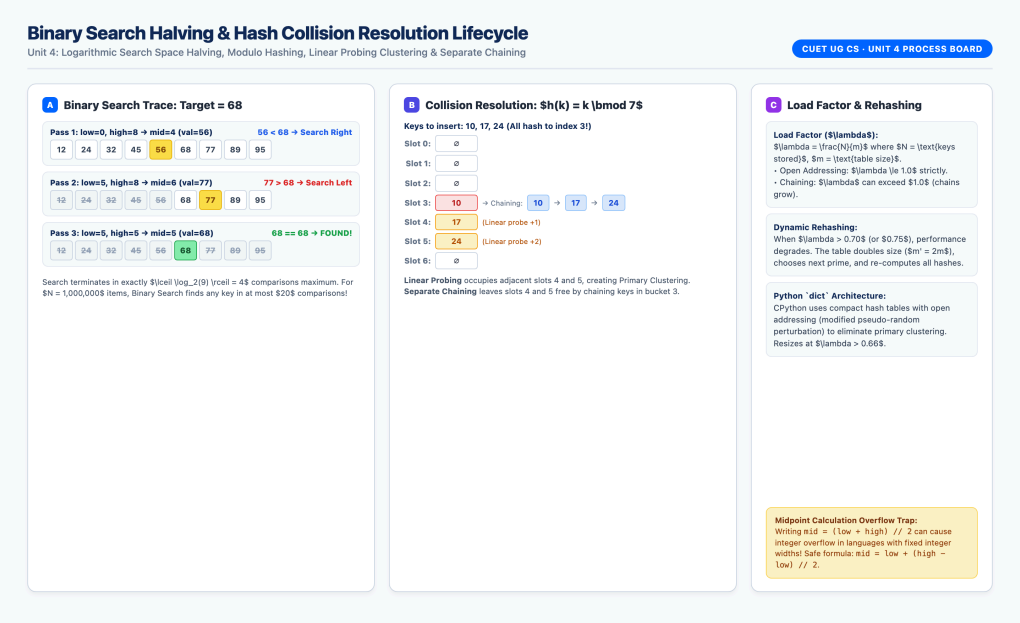


Figure 2: Figure 2: Binary Search Halving & Hash Collision Resolution Lifecycle: Logarithmic midpoint reduction and collision handling via Separate Chaining and Linear Probing.

Concept Module 4: Collision Resolution Strategies

4.1 The Collision Phenomenon (Pigeonhole Principle)

A **Collision** occurs when two distinct keys $k_1 \neq k_2$ map to the exact same hash slot: $h(k_1) = h(k_2)$. By Dirichlet's Pigeonhole Principle, if the universe of keys is larger than table capacity ($|U| > m$), collisions are mathematically inevitable.

4.2 Separate Chaining (Open Hashing)

In **Separate Chaining**, each slot in the hash table points to an independent linked list (or dynamic array) termed a **bucket**:

- When key k hashes to slot $h(k)$, it is appended to the linked list at `table[h(k)]`.
- **Advantages:**
  - Simple implementation; deletion is straightforward (standard linked list node removal).
  - The table never becomes physically full; the load factor λ can easily exceed 1.0.
- **Disadvantages:**
  - Dynamic pointer allocation increases memory overhead and exhibits poor CPU cache locality.
  - **Worst-Case Degradation:** If all N keys hash to the exact same slot, the chain degenerates into an N-element linked list, degrading search time from $O(1)$ to $O(N)$!

4.3 Open Addressing (Closed Hashing)

In **Open Addressing**, all elements are stored directly inside the hash table array itself (no external linked lists). Every slot contains either a key or an empty marker:

- If slot $h(k)$ is occupied, the algorithm calculates alternative probe slots systematically until an empty slot is encountered.

- **Strict Capacity Invariant:** The number of stored keys N can never exceed table size m ($\lambda \leq 1.0$).

1. Linear Probing

The simplest open addressing method probes consecutive adjacent slots sequentially:

$$h(k,i) = (h(k) + i) \bmod m \quad \text{for } i = 0, 1, 2, ..., m-1$$

- **Primary Clustering Hazard:** As slots fill up, contiguous blocks of occupied cells form. Any key hashing anywhere into the cluster must traverse the entire block, lengthening clusters and drastically slowing search time.

2. Quadratic Probing

To eliminate primary clustering, Quadratic Probing spaces out probe steps quadratically:

$$h(k,i) = (h(k) + c_1 i + c_2 i^2) \bmod m$$

- **Secondary Clustering:** Keys that produce the identical initial hash $h(k)$ will still trace the exact same probe sequence.

3. Double Hashing

The most effective open addressing strategy utilizes **two independent hash functions** $h_1(k)$ and $h_2(k)$:

$$h(k,i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

- $h_1(k)$ determines the initial starting slot.
- $h_2(k)$ determines the **probe step size**.
- **Rule:** $h_2(k)$ must **never** evaluate to 0, and $h_2(k)$ must be relatively prime to m (typically achieved by picking prime m and $h_2(k) = 1 + (k \bmod (m-1))$).
- Completely eliminates both primary and secondary clustering.

4.4 Deletion in Open Addressing: The Tombstone Requirement

In open addressing, deleting a key by simply resetting the slot to `None` / empty **breaks search chains**:

- If key k_2 collided with key k_1 and probed past k_1's slot to slot S, later deleting k_1 would leave an empty slot.
- When searching for k_2, the algorithm encounters the empty slot, erroneously assumes k_2 does not exist, and aborts search prematurely!
- **Solution:** Deletions must place a special sentinel marker termed a **Tombstone** (or `DELETED` flag). Searches treat tombstones as occupied slots (continuing the probe), while insertions treat tombstones as free slots available for reuse.

Concept Module 5: Load Factor & Rehashing Mechanics

5.1 Load Factor Definition & Invariants

The **Load Factor** (λ) quantifies the occupancy density of a hash table:

$$\lambda = \frac{N}{m} = \frac{\text{Total Keys Stored}}{\text{Total Table Slots}}$$

- **Separate Chaining:** λ can be any positive value ($\lambda < 1, \lambda = 1, \lambda > 1$). Average chain length equals λ.
- **Open Addressing:** λ is strictly bounded by $\lambda \leq 1.0$. As $\lambda \to 1.0$, the average number of probes required for a successful search increases dramatically:

$$\text{Average Probes (Linear Probing)} \approx \frac{1}{2}\left(1 + \frac{1}{1-\lambda}\right)$$

  - At $\lambda = 0.5$: average probes ≈ 1.5.
  - At $\lambda = 0.9$: average probes ≈ 5.5.

5.2 Dynamic Rehashing Protocol

When λ exceeds a designated threshold (typically 0.70 or 0.75 in general computer science, and 0.66 in CPython):

1. Allocate a new hash table of roughly **double** the capacity ($m' \approx 2m$), choosing the next prime number.
2. Iterate through every active entry in the old table.
3. Re-compute the new hash address $h'(k) = k \bmod m'$ for every key and insert it into the new table.
4. Release the old memory buffer.

- Although rehashing takes $O(N)$ time, its amortized execution cost per insertion remains strictly $O(1)$.

Concept Module 6: CPython Internal Hash Table Architecture

6.1 The Modern Compact Dict Layout (Python 3.6+)

Historically, Python dictionaries stored a sparse array of 24-byte structs (`hash`, `key`, `value`), wasting up to 66% of table space on empty entries. In Python 3.6+ (designed by Raymond Hettinger and implemented by Inada Naoki):

- **Dense Entries Array:** An array `entries` storing tuples `[hash, key, value]` packed contiguously in exact insertion order.
- **Sparse Index Array:** A compact array `indices` of small integer bytes (int8, int16) mapping hash slots to indices in the `entries` array.
- **Benefits:**
  1. Reduces overall memory consumption by 30% to 40%.
  2. Guarantees deterministic **insertion-order iteration** as an official language guarantee!

6.2 SipHash & Hash Randomization Security

To defend against **Hash-DoS attacks** (where malicious web clients submit specially crafted form inputs that all collide into the same bucket, freezing servers at $O(N^2)$ CPU time):

- Python hashes strings using **SipHash**, seeded with a secret per-process random key generated at interpreter startup.
- As a consequence, `hash("string")` produces differing numerical outputs across separate Python process invocations.

Concept Module 7: Comprehensive Worked Examples

WORKED EXAMPLE 1: BINARY SEARCH RANGE HALVING TRACE

A sorted array is given as `arr = [15, 23, 38, 44, 52, 67, 79, 88, 94]`. Trace the sequence of `low`, `high`, and `mid` indices when searching for target key `79`.
Step-by-Step Search Trace:

- Initial indices: `low = 0`, `high = 8`.
- **Iteration 1:**
  - `mid = 0 + (8 - 0) // 2 = 4`.
  - `arr[4] = 52`.
  - Since `52 < 79`, target lies in right half: `low = mid + 1 = 5`.
- **Iteration 2:**
  - `low = 5`, `high = 8`.
  - `mid = 5 + (8 - 5) // 2 = 6`.
  - `arr[6] = 79`.
  - Match found! Search terminates successfully at index `6`.
- Total comparisons: exactly 2 iterations.

WORKED EXAMPLE 2: HASH TABLE INSERTION WITH LINEAR PROBING

Given a hash table of size $m = 7$ and hash function $h(k) = k \bmod 7$. Insert keys `[12, 19, 26, 33, 40]` using Open Addressing with Linear Probing. Show the final slot occupancy of the table.
Step-by-Step Insertion:

1. Key `12`: $h(12) = 12 \bmod 7 = 5$. Slot 5 is empty → `table[5] = 12`.
2. Key `19`: $h(19) = 19 \bmod 7 = 5$. Slot 5 is occupied → probe slot $(5 + 1) \bmod 7 = 6$. Empty → `table[6] = 19`.
3. Key `26`: $h(26) = 26 \bmod 7 = 5$. Slot 5 occupied → probe 6 (occupied) → probe $(6 + 1) \bmod 7 = 0$. Empty → `table[0] = 26`.
4. Key `33`: $h(33) = 33 \bmod 7 = 5$. Probes 5, 6, 0 (all occupied) → probe 1. Empty → `table[1] = 33`.
5. Key `40`: $h(40) = 40 \bmod 7 = 5$. Probes 5, 6, 0, 1 (occupied) → probe 2. Empty → `table[2] = 40`.

Final Hash Table:

- Slot 0: `26`, Slot 1: `33`, Slot 2: `40`, Slot 3: `None`, Slot 4: `None`, Slot 5: `12`, Slot 6: `19`.

Analysis: Demonstrates extreme Primary Clustering where all 5 keys collided at index 5 and formed a contiguous block wrapping around to slot 2.

WORKED EXAMPLE 3: SEPARATE CHAINING BUCKET INSERTION

Using the same keys `[12, 19, 26, 33, 40]` and $m = 7$ with $h(k) = k \bmod 7$, show the hash table layout under Separate Chaining.
Layout:

- Slots 0, 1, 2, 3, 4, 6: Empty (`None`).
- Slot 5: Points to linked list `12 -> 19 -> 26 -> 33 -> 40 -> None`.

Load Factor: $\lambda = \frac{5}{7} \approx 0.714$. Slots 0 to 4 remain completely unoccupied while slot 5 carries all load.

WORKED EXAMPLE 4: DOUBLE HASHING COLLISION RESOLUTION NUMERICAL TRACE

A hash table has size $m = 11$. Hash functions are defined as:

$$h_1(k) = k \bmod 11 \quad \text{and} \quad h_2(k) = 1 + (k \bmod 7)$$

Insert keys `58` and `80`. Trace their initial hashes and probe sequence.

Trace:

1. Key `58`:
   - $h_1(58) = 58 \bmod 11 = 3$. Slot 3 is empty → `table[3] = 58`.
2. Key `80`:
   - $h_1(80) = 80 \bmod 11 = 3$. Collision at slot 3!
   - Step size: $h_2(80) = 1 + (80 \bmod 7) = 1 + 3 = 4$.
   - First probe: $h(80, 1) = (3 + 1 \cdot 4) \bmod 11 = 7 \bmod 11 = 7$.
   - Slot 7 is empty → `table[7] = 80`.

Comparison: Linear probing would have probed slot 4; Double Hashing jumped directly to slot 7, completely bypassing adjacent cluster slots.

WORKED EXAMPLE 5: FIRST & LAST OCCURRENCE OF TARGET IN O(LOG N) TIME

Implement a function `find_occurrences(arr, target)` that returns the first and last index of `target` in a sorted list containing duplicate integers in strictly $O(\log N)$ time:

```python
def find_first(arr, target):
    low, high, ans = 0, len(arr) - 1, -1
    while low <= high:
        mid = low + (high - low) // 2
        if arr[mid] == target:
            ans = mid
            high = mid - 1  # Keep searching left
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return ans

def find_last(arr, target):
    low, high, ans = 0, len(arr) - 1, -1
    while low <= high:
        mid = low + (high - low) // 2
        if arr[mid] == target:
            ans = mid
            low = mid + 1   # Keep searching right
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return ans

nums = [2, 4, 4, 4, 7, 7, 9]
print(find_first(nums, 4), find_last(nums, 4))  # Output: 1, 3
```

WORKED EXAMPLE 6: TWO-SUM LOOKUP IN O(N) TIME USING HASH MAP

Given an array `nums = [2, 7, 11, 15]` and `target = 9`, find the indices of the two numbers that sum to target using a Hash Table in $O(N)$ time:

```python
def two_sum(nums, target):
    seen = {}  # Maps value -> index
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []

print(two_sum([2, 7, 11, 15], 9))  # Output: [0, 1]
```

Analysis: Checking `complement in seen` takes $O(1)$ expected time. Total runtime is $O(N)$, drastically outperforming $O(N^2)$ nested loops.

WORKED EXAMPLE 7: IMPLEMENTING OPEN ADDRESSING HASH TABLE WITH TOMBSTONES

Write a Python class `LinearProbingHashTable` supporting `insert`, `search`, and `delete` using tombstones:

```python
class LinearProbingHashTable:
    TOMBSTONE = object()
```

```
    def __init__(self, size=11):
        self.size = size
        self.table = [None] * size

    def _hash(self, key):
        return hash(key) % self.size

    def insert(self, key):
        slot = self._hash(key)
        first_tombstone = None

        for i in range(self.size):
            curr = (slot + i) % self.size
            if self.table[curr] is None:
                dest = first_tombstone if first_tombstone is not None else curr
                self.table[dest] = key
                return True
            elif self.table[curr] is self.TOMBSTONE and first_tombstone is None:
                first_tombstone = curr
            elif self.table[curr] == key:
                return False  # Duplicate
        if first_tombstone is not None:
            self.table[first_tombstone] = key
            return True
        raise OverflowError("Hash table full")

    def delete(self, key):
        slot = self._hash(key)
        for i in range(self.size):
            curr = (slot + i) % self.size
            if self.table[curr] is None:
                return False
            elif self.table[curr] == key:
                self.table[curr] = self.TOMBSTONE
                return True
        return False
```

Concept Module 8: Advanced Search Variations & Hashing Systems

8.1 Specialized Search Paradigms: Interpolation & Ternary Search

Beyond canonical Linear and Binary Search, specialized algorithms leverage data distribution properties:

1. Interpolation Search (Uniformly Distributed Datasets)

When numeric elements in a sorted array are uniformly distributed (e.g., telephone directory names or evenly spaced timestamps), Interpolation Search mimics human search behavior by estimating the target location based on value magnitude:

$$\text{probe} = \text{low} + \left\lfloor \frac{\text{target} - \text{arr[low]}}{\text{arr[high]} - \text{arr[low]}} \cdot (\text{high} - \text{low}) \right\rfloor$$

- **Complexity:** Achieves remarkable $O(\log \log N)$ average-case search time!
- **Hazard:** If data distribution is heavily skewed or non-uniform, runtime degrades to catastrophic $O(N)$ linear time.

2. Ternary Search vs Binary Search

Ternary Search splits the sorted search space into three equal segments using two midpoints $\texttt{mid}_1 = \text{low} + \frac{\text{high} - \text{low}}{3}$ and $\texttt{mid}_2 = \text{high} - \frac{\text{high} - \text{low}}{3}$:

- Recurrence: $T(N) = T\left(2\frac{N}{3}\right) + 2$.
- **Theoretical Comparison:** Although the search range shrinks by a factor of $\frac{2}{3}$, each step requires 2 comparisons.

$$\text{Total Comparisons} = 2 \cdot \log_3(N) = \frac{2}{\log_2(3)} \log_2(N) \approx 1.2618 \log_2(N)$$

- Consequently, Ternary Search performs roughly 26% **more comparisons** than Binary Search! It is primarily utilized for locating unimodal function extrema rather than standard array search.

8.2 Universal & Perfect Hashing Architecture

When static dictionaries (such as language compilers lookup tables for reserved keywords) require guaranteed $O(1)$ worst-case lookup:

1. Universal Hashing

To prevent malicious attackers from picking keys that collide in a fixed hash function, **Universal Hashing** selects a hash function h at random from a carefully designed universal family $\mathcal{H}$ at runtime:

- Guarantees that for any two distinct keys, the collision probability is at most $\frac{1}{m}$, ensuring $O(1)$ expected time regardless of input data.

2. Perfect Hashing (The FKS Scheme)

Developed by Fredman, Komlós, and Szemerédi (1984), Perfect Hashing achieves **strictly $O(1)$ worst-case search time** with zero collisions:

- **Two-Level Hashing:**
  1. **Primary Table:** Size $m = N$, using universal hash h. Collisions occur at primary slots.
  2. **Secondary Tables:** For each slot i containing n_i colliding keys, allocate a secondary hash table of size $s_i = n_i^2$.
     - By the Birthday Paradox, a table of size n_i^2 can store n_i keys with zero collisions with probability $> \frac{1}{2}$.
     - Total allocated space across all secondary tables is guaranteed $O(N)$.

8.3 Additional Comprehensive Worked Examples

WORKED EXAMPLE 8: SEARCH IN A ROTATED SORTED ARRAY IN O(LOG N) TIME

An array sorted in ascending order is rotated at an unknown pivot (e.g. `[4, 5, 6, 7, 0, 1, 2]`). Implement a modified binary search to locate `target` in strictly $O(\log N)$ time:

```
def search_rotated(nums, target):
    low, high = 0, len(nums) - 1
    while low <= high:
        mid = low + (high - low) // 2
        if nums[mid] == target:
            return mid

        # Determine which half is normally sorted
        if nums[low] <= nums[mid]:  # Left half is sorted
            if nums[low] <= target < nums[mid]:
                high = mid - 1
            else:
                low = mid + 1
        else:                       # Right half is sorted
            if nums[mid] < target <= nums[high]:
                low = mid + 1
            else:
                high = mid - 1
    return -1

arr = [4, 5, 6, 7, 0, 1, 2]
print(search_rotated(arr, 0))  # Output: 4
print(search_rotated(arr, 3))  # Output: -1
```

Analysis: In any rotated sorted array, at least one half (left or right of `mid`) is always normally sorted. Checking the sorted boundary permits halving the search space in $O(\log N)$ time.

WORKED EXAMPLE 9: BINARY SEARCH ON SOLUTION SPACE (SQUARE ROOT)

Implement `integer_sqrt(n)` to calculate $\lfloor \sqrt{n} \rfloor$ for a non-negative integer n in $O(\log n)$ time without utilizing external floating-point math libraries:

```
def integer_sqrt(n):
    if n < 2:
        return n
    low, high, ans = 1, n // 2, 1
    while low <= high:
        mid = low + (high - low) // 2
        sq = mid * mid
        if sq == n:
            return mid
        elif sq < n:
            ans = mid       # Potential answer
            low = mid + 1  # Try larger
        else:
            high = mid - 1 # Try smaller
    return ans

print(integer_sqrt(16))  # Output: 4
print(integer_sqrt(27))  # Output: 5 (since 5^2 = 25 <= 27)
```

WORKED EXAMPLE 10: QUADRATIC PROBING STEP-BY-STEP INDEX CALCULATION

Given a hash table of size $m = 11$, primary hash function $h(k) = k \bmod 11$, and quadratic probe formula:

$$h(k, i) = (h(k) + i^2) \bmod 11$$

Insert keys `[22, 33, 44]` into an initially empty table:

- Key `22`: $h(22) = 22 \bmod 11 = 0$. Slot 0 is empty → `table[0] = 22`.
- Key `33`: $h(33) = 33 \bmod 11 = 0$. Collision at slot 0!
  - Probe $i = 1$: $(0 + 1^2) \bmod 11 = 1$. Slot 1 is empty → `table[1] = 33`.
- Key `44`: $h(44) = 44 \bmod 11 = 0$. Collision at slot 0!

- Probe $i = 1$: $(0 + 1^2) \bmod 11 = 1$ (Occupied by 33).
- Probe $i = 2$: $(0 + 2^2) \bmod 11 = 4$. Slot 4 is empty → `table[4] = 44`.

Final Slots: Slot 0: `22`, Slot 1: `33`, Slot 4: `44`. Notice that slot 2 and slot 3 were skipped, preventing contiguous primary clustering.

OurMocks

Master Mindmap: Searching & Hashing

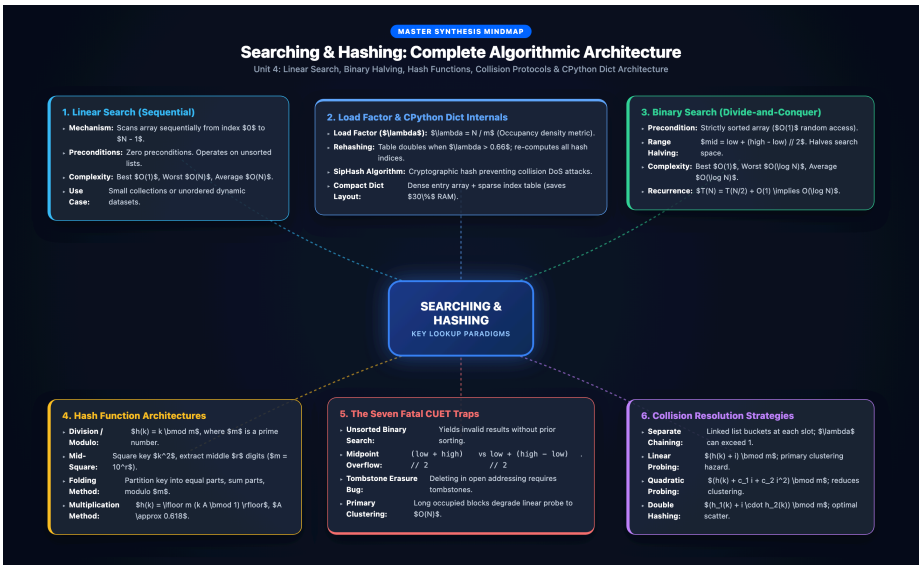


Figure 3: Figure 3: Master Synthesis Mindmap: Structural framework of Unit 4 Searching & Hashing Techniques.

OurMocks

High-Yield Cheatsheet & Exam Traps

MASTER SEARCHING & HASHING REVISION CHEATSHEET

- **Linear Search:** Operates on unsorted or sorted data; sequentially checks every index; Best: $O(1)$, Worst: $O(N)$, Average: $O(N)$.
- **Binary Search:** Requires **strictly sorted** list with $O(1)$ random access; halves search space every step; Best: $O(1)$, Worst: $O(\log N)$, Average: $O(\log N)$.
- **Max Binary Search Comparisons:** $\lfloor \log_2 N \rfloor + 1$. For $N = 1,000,000$, maximum comparisons is exactly 20.
- **Midpoint Safe Equation:** `mid = low + (high - low) // 2` prevents integer arithmetic overflow.
- **Hash Functions:** Deterministic mapping $h(k) = k \bmod m$. Modulo m must be a prime number to guarantee uniform scatter.
- **Separate Chaining:** Open hashing; each slot holds a linked list bucket; λ can exceed 1; degrades to $O(N)$ if all keys collide.
- **Open Addressing:** Closed hashing; elements stored directly in table; $\lambda \leq 1.0$; uses probing sequences to resolve collisions.
- **Linear Probing:** $h(k, i) = (h(k) + i) \bmod m$. Suffers from Primary Clustering (long contiguous blocks of occupied cells).
- **Double Hashing:** $h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$. Eliminates both primary and secondary clustering; $h_2(k) \neq 0$.
- **Tombstone Invariant:** Deletion in open addressing must leave a sentinel marker; resetting to `None` breaks search probe chains.
- **Load Factor (λ):** $\lambda = \frac{N}{m}$. Rehash dynamically when $\lambda > 0.66$ to 0.75 by doubling table size to next prime.
- **CPython Dict:** Uses compact dense table + sparse index array (saves 30% memory); uses SipHash random seed to prevent Hash-DoS.

THE SEVEN FATAL CUET TRAPS IN SEARCHING & HASHING

1. **Binary Search Unsorted Array Trap:** Running Binary Search on an unsorted list produces invalid results without raising an error! If list is unsorted and searched only once, Linear Search ($O(N)$) is strictly faster than sorting ($O(N \log N)$) + binary search.
2. **The Midpoint Calculation Overflow Trap:** Calculating `mid = (low + high) // 2` causes integer overflow in fixed-width systems. Always use `low + (high - low) // 2`.
3. **Tombstone Deletion Erasure Trap:** In Open Addressing, setting `table[slot] = None` upon deletion severs the probing trail! Any key that probed past that slot becomes permanently unreachable. A tombstone marker is mandatory.
4. **Double Hashing Modulo Zero Trap:** The second hash function $h_2(k)$ must **never** return 0! If $h_2(k) = 0$, the probe step is 0, trapping the algorithm in an infinite loop on the same slot.
5. **Linear Probing Primary Clustering Trap:** Assuming linear probing distributes collisions evenly. In reality, large clusters act as collision magnets, pulling nearby probes into the cluster and degrading lookup towards $O(N)$.
6. **Load Factor Range Trap:** Believing λ must always be ≤ 1. In Separate Chaining, λ can easily exceed 1.0 (e.g. 100 keys in 10 buckets gives $\lambda = 10$). Only Open Addressing strictly requires $\lambda \leq 1.0$.
7. **Hash Collision Worst-Case Complexity Trap:** Assuming hash tables guarantee $O(1)$ search time. In the worst case (where all N keys collide), both Chaining and Linear Probing degrade to $O(N)$ linear time!

CUET UG Exam-Style Practice Questions (15 MCQs)

Q1. What is the maximum number of comparisons required to search for an element in a sorted list of 1000 elements using Binary Search?

(A) 10 **(B)** 500
(C) 1000 **(D)** 1

Sol. Q1. In Binary Search, the maximum number of comparisons in the worst case is given by $\lfloor \log_2 N \rfloor + 1$. For $N = 1000$: $\lfloor \log_2 1000 \rfloor + 1 = \lfloor 9.965 \rfloor + 1 = 9 + 1 = 10$ comparisons.
Answer: (A)

Q2. Which of the following is a strict mandatory precondition for executing a Binary Search algorithm on a collection?

(A) The collection must be stored in a doubly-linked list **(B)** The collection must be sorted and support constant-time random access
(C) The collection size must be an exact power of two **(D)** All elements in the collection must be unique integers

Sol. Q2. Binary Search requires that the elements be strictly sorted (in ascending or descending order) and that the underlying data structure allow $O(1)$ direct index access (like an array or Python list) to inspect midpoints.
Answer: (B)

Q3. In Open Addressing with Linear Probing, the tendency of occupied hash table slots to form long contiguous blocks that degrade lookup efficiency is known as:

(A) Secondary Clustering **(B)** Chaining Drift
(C) Primary Clustering **(D)** Hash-DoS Degradation

Sol. Q3. Primary Clustering is the phenomenon in linear probing where occupied slots coalesce into large contiguous runs. Any key whose initial hash maps anywhere within the block must probe through the entire cluster, compounding search latency.
Answer: (C)

Q4. Given a hash table of size $m = 10$ and hash function $h(k) = k \bmod 10$. The keys `42`, `52`, and `62` are inserted using Linear Probing. In which slot will the key `62` be placed?

(A) Slot 2 **(B)** Slot 3
(C) Slot 5 **(D)** Slot 4

Sol. Q4. Tracing insertions:

1. Key `42`: $h(42) = 42 \bmod 10 = 2$. Slot 2 is empty → stored in Slot 2.
2. Key `52`: $h(52) = 52 \bmod 10 = 2$. Collision at slot 2! Probe $(2 + 1) \bmod 7 = 3$. Empty → stored in Slot 3.
3. Key `62`: $h(62) = 62 \bmod 10 = 2$. Collision at 2 (occupied) → probe 3 (occupied) → probe $(3 + 1) \bmod 10 = 4$. Empty → stored in Slot 4.

Answer: (D)

Q5. Why must the table size m in the Division Hash method $h(k) = k \bmod m$ preferably be chosen as a prime number?

(A) Prime numbers reduce common factor collisions across arithmetic sequences of keys **(B)** Prime numbers guarantee that no collisions can ever occur
(C) Prime numbers allow modulo calculation to be done in $O(\log N)$ time **(D)** Python syntax strictly requires prime table sizes for dictionary hashing

Sol. Q5. Choosing a prime table size m ensures that keys sharing common factors with typical data intervals (such as multiples of powers of 2 or 10) scatter evenly across all buckets rather than clustering into a small subset of indices.
Answer: (A)

CUET UG Exam-Style Practice Questions (Part 2)

Q6. What is the worst-case time complexity of searching for an element in a Hash Table of N keys using Separate Chaining?

(A) $O(1)$ constant time
(B) $O(N)$ linear time
(C) $O(\log N)$ logarithmic time
(D) $O(N \log N)$

Sol. Q6. In the worst case (where a pathological hash function maps every single key to the exact same hash slot), all N keys are appended into a single linked list bucket, degrading lookup to a linear search of $O(N)$ time.
Answer: (B)

Q7. Why is writing `mid = low + (high - low) // 2` preferred over `mid = (low + high) // 2` in binary search implementations?

(A) It calculates the midpoint in $O(1)$ instead of $O(\log N)$
(B) It works on floating-point lists whereas the other does not
(C) It prevents potential integer overflow when `low + high` exceeds maximum integer capacity
(D) It automatically sorts the input array during midpoint calculation

Sol. Q7. In languages with fixed-width integers, adding `low + high` can exceed the maximum signed 32-bit integer limit ($2^{\{31\}} - 1$), causing arithmetic overflow and wrapping to a negative value. `low + (high - low) // 2` is mathematically equivalent and overflow-safe.
Answer: (C)

Q8. When deleting a key from a hash table implemented using Open Addressing, why is a special Tombstone marker placed instead of resetting the slot to `None`?

(A) To prevent memory leak exceptions in CPython runtime
(B) To maintain sorted order among hashed keys
(C) To dynamically double the table size upon every deletion
(D) To avoid breaking the probing chain for keys that collided and probed past this slot

Sol. Q8. If an open-addressing slot is simply reset to `None`, subsequent searches for keys that probed past that slot will terminate prematurely upon seeing `None`, falsely reporting that the key does not exist. A Tombstone marker preserves the probe chain.
Answer: (D)

Q9. What is the Load Factor (λ) of a hash table with $m = 20$ slots currently storing 15 elements?

(A) 0.75
(B) 1.33
(C) 0.66
(D) 5

Sol. Q9. The Load Factor is defined as $\lambda = \frac{N}{m}$. With $N = 15$ keys and $m = 20$ slots, $\lambda = \frac{15}{20} = 0.75$.
Answer: (A)

Q10. In Double Hashing with probe formula $h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$, what critical constraint must be satisfied by the secondary hash function $h_2(k)$?

(A) $h_2(k)$ must always evaluate to zero
(B) $h_2(k)$ must never evaluate to zero and should be relatively prime to m
(C) $h_2(k)$ must equal $h_1(k)$ squared
(D) $h_2(k)$ must return a negative integer

Sol. Q10. If $h_2(k) = 0$, the step size is 0, trapping subsequent probes in an infinite loop on the original slot. Additionally, $h_2(k)$ must be relatively prime to m to ensure the probe sequence visits every slot in the table.
Answer: (B)

CUET UG Exam-Style Practice Questions (Part 3)

Q11. A programmer needs to perform a single one-off search for an element in an unsorted array of N items. Which strategy is computationally superior?

(A) Sorting the array using Merge Sort ($O(N \log N)$) and then performing Binary Search ($O(\log N)$)
(B) Building an entire binary search tree and doing in-order traversal
(C) Performing a direct Linear Search ($O(N)$)
(D) Hashing all elements into a new table and querying the key

Sol. Q11. For a single search, Linear Search runs in $O(N)$ time. Sorting first takes $O(N \log N)$ time plus $O(\log N)$ for the search, which is strictly worse than $O(N)$ for all $N > 1$. Sorting is only justified if multiple repeated queries are conducted.
Answer: (C)

Q12. Which cryptographic hash algorithm is utilized internally by CPython to protect dictionary hashing against collision Denial-of-Service (Hash-DoS) attacks?

(A) SHA-256
(B) MD5
(C) CRC32
(D) SipHash

Sol. Q12. CPython utilizes the SipHash algorithm, keyed with a secret random seed generated per interpreter process, to ensure string hashes cannot be predicted by external attackers attempting to engineer worst-case collision DoS attacks.
Answer: (D)

Q13. What is the recurrence relation that governs the runtime of the recursive Binary Search algorithm on an array of size N?

(A) $T(N) = T\left(\frac{N}{2}\right) + O(1)$
(B) $T(N) = 2T\left(\frac{N}{2}\right) + O(N)$
(C) $T(N) = T(N-1) + O(1)$
(D) $T(N) = 2T\left(\frac{N}{2}\right) + O(1)$

Sol. Q13. In recursive Binary Search, one comparison of $O(1)$ time is made at the midpoint, after which only one half-sized subproblem ($\frac{N}{2}$) is recursively examined. Hence, $T(N) = T\left(\frac{N}{2}\right) + O(1)$, which solves to $O(\log N)$.
Answer: (A)

Q14. In the Mid-Square hash method, if key $k = 32$, and the middle 2 digits of k^2 are selected as the hash index, what is the resulting slot address?

(A) 32
(B) 02
(C) 24
(D) 10

Sol. Q14. Calculating the square: $k^2 = 32^2 = 1024$. The four-digit number is 1024. The middle two digits are `02`. Therefore, the resulting hash address is 02 (or 2).
Answer: (B)

Q15. Which of the following built-in Python collections requires that all stored keys/elements be strictly hashable and immutable?

(A) Python `list` and `bytearray`
(B) Python `dict` values and `list` elements
(C) Python `set` elements and `dict` keys
(D) Python `deque` and `queue.Queue`

Sol. Q15. Python `set` elements and `dict` keys must implement the `__hash__()` method and be immutable (e.g. `int`, `str`, `tuple`). Mutable objects like lists or dictionaries cannot serve as set elements or dictionary keys.
Answer: (C)